

Δελατόλας Γρηγόριος (03119202)
Κυριάκος Γεώργιος (03119204)
Ομάδα: oslaba74

Άσκηση 1.1

main.c

```
#include "myzing.h"
int main()
{
    zing();
    return 0;
}
```

zing2.c

```
#include <unistd.h>
#include <stdio.h>

void zing()
{
    printf ("How are you?, ");
    printf (getlogin());
    printf ("\n");
}
```

Makefile

all: myzing zing2

myzing: myzing.o main.o
gcc main.o myzing.o -o myzing

zing2: zing2.o main.o
gcc main.o zing2.o -o zing2

main.o: main.c
gcc -Wall -c main.c

Έξοδος εκτέλεσης για το εκτελέσιμο myzing : Hello, oslaba74

Έξοδος εκτέλεσης για το εκτελέσιμο zing2 : How are you?, oslaba74

Ερώτηση 1: Ποιο σκοπό εξυπηρετεί η επικεφαλίδα;

Απάντηση

Το header file περιέχει τον τύπο της συνάρτησης και τα ορίσματα της (πχ `int sum(int a, int b);`)

Με τη χρήση του header file αντιγράφεται το περιεχόμενο του header file στον κώδικα προς εκτέλεση (το αρχείο στο οποίο γίνεται `include`).

Έτσι, σε περίπτωση που χρειαστούμε το περιεχόμενο του header file σε πολλά αρχεία κώδικα δεν χρειάζεται να μεταφέρουμε ολόκληρο τον κώδικα ο οποίος ενδέχεται να έχει μεγάλη έκταση, δυσχαιρένοντας τη διαδικασία γραφής.

Επιπροσθέτως, ο λόγος που χρησιμοποιείται το header file είναι για την περίπτωση κατά την οποία αλλάζουν τα ορίσματα της συνάρτησης. Με χρήση του header file δεν χρειάζεται να γίνει η ίδια αλλαγή και στον κώδικα προς εκτέλεση, στην περίπτωση που χρησιμοποιούμε την ίδια συνάρτηση σε πολλά αρχεία.

Ερώτηση 4: Αντιμετώπιση προβλήματος 500 συναρτήσεων

Απάντηση

Ένας τρόπος για να επιλύσουμε το πρόβλημα είναι να μεταφέρουμε τη συνάρτηση σε ένα ξεχωριστό αρχείο το οποίο θα συνδέεται με το αρχικό μέσω ενός header file. Στη συνέχεια θα φτιάξουμε ένα καινούργιο target μέσα στο makefile το οποίο θα έχει ως dependency μόνο το αρχείο της συνάρτησης. Έτσι, η αλλαγή της συνάρτησης θα οδηγήσει στη μεταγλώττιση μόνο του κώδικα που αφορά τη συνάρτηση και όχι ολόκληρου.

Ερώτηση 5: Χάθηκε το `foo.c` ?

Απάντηση

Στη συγκεκριμένη εντολή το πρώτο `foo.c` δεξιά του `-o` είναι το όνομα του εκτελέσιμου αρχείου που θα παραχθεί από την μεταγλώττιση του δεύτερου `foo.c`. Έτσι, ο κώδικας του εκτελέσιμου θα αποθηκευτεί στο (πρώτο) `foo.c`. Τελικά, θα έχουμε μόνο ένα εκτελέσιμο αρχείο το οποίο έχει γλώσσα μηχανής και όχι τον κώδικα μας.

Ουσιαστικά βλέπουμε το περιεχόμενο ενός εκτελέσιμου αρχείου.

Άσκηση 1.2

doWrite.c

```
#include <unistd.h>
#include <stdio.h>
#include <string.h>
```

```
void doWrite(int fd, const char *buff, int len)
{
    size_t idx = 0;
    ssize_t wcnt;
    do
    {
        wcnt = write(fd, buff + idx, len - idx);
        if (wcnt == -1){ perror("write"); return;}
        idx += wcnt;
    } while (idx < len);
}
```

write_file.c

```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

```
#include <fcntl.h>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <string.h>
```

```
#include "doWrite.h"
```

```
void write_file(int fd, const char *infile)
```

```
{
```

```
    char buff[1024];
```

```
    ssize_t rcnt;
```

```
    int fd_source = open(infile, O_RDONLY);
```

```
    if (fd_source == -1) { perror("open"); exit(1); }
```

```
    for (;;) // copy infile to buffer and then to fd
```

```
    {
```

```
        rcnt = read(fd_source, buff, sizeof(buff) - 1); // copy infile to buffer
```

```
        if (rcnt == 0) return;
```

```
        if (rcnt == -1){ perror("read"); return; }
```

```
        buff[rcnt] = '\0';
```

```
        int len = strlen(buff);
```

```
        doWrite(fd, buff, len); // copy buffer to fd
```

```
    }
```

```
    close(fd_source);
```

```
    return;
```

```
}
```

fconc.c

```
#include "write_file.h"
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

int main (int argc , char **argv){
    if (argc < 3 || argc > 4){
        printf("we need 2 or 3 files! \n");
    }
    if (argc == 4){
        int fd, oflags, mode;
        oflags = O_CREAT | O_WRONLY | O_TRUNC;
        mode = S_IRUSR | S_IWUSR;
        fd = open(argv[3], oflags, mode);
        if (fd == -1){perror("open");exit(1);}

        write_file(fd,argv[1]);
        write_file(fd,argv[2]);
        close(fd);
        return 0;
    }
    if (argc == 3){
        int fd, oflags, mode;
        oflags = O_CREAT | O_WRONLY | O_TRUNC;
        mode = S_IRUSR | S_IWUSR;
        fd = open("fconc.out", oflags, mode);
        if (fd == -1){perror("open");exit(1);}

        write_file(fd,argv[1]);
        write_file(fd,argv[2]);
        close(fd);
        return 0;
    }
}
```

Ερώτηση 1

Εκτελώντας την εντολή : `strace -c ls /test`

```
-----  
0.00  0.000000      0   11    read  
0.00  0.000000      0    4    write  
0.00  0.000000      0   15    4 open  
0.00  0.000000      0   13    close  
0.00  0.000000      0    1    1 stat  
0.00  0.000000      0   11    fstat  
0.00  0.000000      0    1    1 lstat  
0.00  0.000000      0   26    mmap  
0.00  0.000000      0   16    mprotect  
0.00  0.000000      0    3    munmap  
0.00  0.000000      0    3    brk  
0.00  0.000000      0    2    rt_sigaction  
0.00  0.000000      0    1    rt_sigprocmask  
0.00  0.000000      0    2    ioctl  
0.00  0.000000      0    9    9 access  
0.00  0.000000      0    1    execve  
0.00  0.000000      0    1    getrlimit  
0.00  0.000000      0    2    2 statfs  
0.00  0.000000      0    1    arch_prctl  
0.00  0.000000      0    1    set_tid_address  
0.00  0.000000      0    1    set_robust_list  
-----  
100.00  0.000000      125   17 total
```

Μας δείχνει τα system calls που πραγματοποιούνται όταν τρέχουμε το εκτελέσιμο test.

Προαιρετικές ερωτήσεις

Ερώτηση 1

Εκτελώντας την εντολή : `strace -o file -c strace -l` έχουμε το εξής αποτέλεσμα μέσα στο file

% time	seconds	usecs/call	calls	errors	syscall
0.00	0.000000	0	1		read
0.00	0.000000	0	2		write
0.00	0.000000	0	2		open
0.00	0.000000	0	2		close
0.00	0.000000	0	2		fstat
0.00	0.000000	0	8		mmap
0.00	0.000000	0	4		mprotect
0.00	0.000000	0	1		munmap
0.00	0.000000	0	3		brk
0.00	0.000000	0	1		rt_sigaction
0.00	0.000000	0	3	3	access
0.00	0.000000	0	1		getpid
0.00	0.000000	0	1		execve
0.00	0.000000	0	1		uname
0.00	0.000000	0	1		arch_prctl
100.00	0.000000		33	3	total

Μας δείχνει τα system calls που πραγματοποιούνται όταν εκτελούμε την εντολή strace.

Ερώτηση 3η

```
#include "write_file.h"
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

int main (int argc , char **argv){
    if (argc < 3){
        printf("we need at least 2 files! \n");
    }
    if (argc == 3){
        int fd, oflags, mode;
        oflags = O_CREAT | O_WRONLY | O_TRUNC;
        mode = S_IRUSR | S_IWUSR;
        fd = open(argv[3], oflags, mode);
        if (fd == -1) { perror("open"); exit(1); }

        write_file(fd,argv[1]);
        write_file(fd,argv[2]);
        close(fd);
        return 0;
    }
    if (argc >= 4){
        int fd, oflags, mode;
        oflags = O_CREAT | O_WRONLY | O_TRUNC;
        mode = S_IRUSR | S_IWUSR;
        fd = open("fconc.out", oflags, mode);
        if (fd == -1){perror("open");exit(1);}
        for(int i = 1; i < argc - 1; i++) write_file(fd, argv[i]);
        close(fd);
        return 0;
    }
}
```